

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÀI TẬP LỚN
Kiến Trúc Máy Tính (CO2007)
FIVE IN A ROW

Giảng viên hướng dẫn: Nguyễn Thành Lộc
Nhóm TN01 - HK242

STT	Họ và tên	MSSV	Ngành	Phân công nhiệm vụ
1	Nguyễn Hoàng Thiên Hậu	2310930	KTMT	Viết code MIPS
2	Nguyễn Việt Hoàng	2311066	KTMT	Viết báo cáo
3	Nguyễn Đăng Khánh	2311512	KHMT	Viết báo cáo

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 5 NĂM 2025

Mục lục

1	Giới Thiệu Đề Tài	2
2	Mục Đích Của Đề Tài	3
3	Phương Pháp Thực Hiện Đề Tài	5
4	Giải Thuật Chính Của Chương Trình	7
5	Hiện Thực Các Hàm Hỗ Trợ (Macro)	9
6	Chạy Chương Trình Trò Chơi Gomoku	44
7	Tổng Kết	55
8	Tài Liệu Tham Khảo	57

1 Giới Thiệu Đề Tài

Trò chơi trên bàn cờ là một phần không thể thiếu trong lịch sử phát triển của nhân loại, từ các trò chơi cổ điển như Cờ vua, Cờ vây cho đến các trò hiện đại như Uno hay Poker. Một trong những trò chơi nổi bật với tính chiến thuật đơn giản nhưng sâu sắc là **Gomoku**, còn được biết đến với tên gọi **“Caro”** tại Việt Nam.

Gomoku là trò chơi trừu tượng có nguồn gốc từ Nhật Bản vào thời kỳ Edo, với tên gọi ban đầu là *gomokunarabe* – trong đó “go” nghĩa là năm, “moku” là đơn vị đếm quân cờ và “narabe” nghĩa là xếp hàng. Trò chơi nhanh chóng lan rộng và phổ biến ở nhiều quốc gia châu Á như Trung Quốc, Hàn Quốc, Việt Nam, và hiện nay được tổ chức thành giải đấu chính thức trên quy mô quốc tế. Luật chơi của Gomoku rất đơn giản: hai người chơi lần lượt đặt quân cờ của mình lên các ô trống trong bảng kích thước 15×15 . Người chơi thứ nhất sử dụng ký hiệu **X**, người thứ hai sử dụng ký hiệu **O**. Người chiến thắng là người đầu tiên tạo thành một dãy liên tiếp gồm **5 quân cờ** của mình theo hàng ngang, hàng dọc, hoặc đường chéo. Nếu toàn bộ bảng đã đầy mà không có người thắng, ván đấu kết thúc với kết quả hoà. Với luật chơi đơn giản nhưng giàu tính chiến lược, Gomoku đã trở thành trò chơi trí tuệ được yêu thích bởi mọi lứa tuổi.

Đề tài này yêu cầu sinh viên hiện thực hóa trò chơi Gomoku trên nền tảng **ngôn ngữ hợp ngữ MIPS** – một kiến trúc vi xử lý phổ biến trong giảng dạy ngành Kiến trúc Máy tính. Bằng cách mô phỏng một trò chơi cổ điển, bài tập không chỉ giúp sinh viên áp dụng kiến thức lý thuyết mà còn phát triển kỹ năng tư duy thuật toán, làm việc nhóm và xử lý giao tiếp với người dùng thông qua giao diện dòng lệnh. Ngoài những chức năng chính của trò chơi, trong đề tài này nhóm còn mở rộng thêm các chức năng khác để trò chơi thêm phong phú và cung cấp nhiều sự lựa chọn cho người chơi.

2 Mục Đích Của Đề Tài

Mục tiêu chính của bài tập lớn là hiện thực một trò chơi **Gomoku** hoàn chỉnh chạy trên trình giả lập **MARS** sử dụng ngôn ngữ **MIPS**. Tuy nhiên, nhóm không chỉ dừng lại ở việc hiện thực chức năng cơ bản mà còn mở rộng và tổ chức mã nguồn một cách rõ ràng, tối ưu. Qua đề tài này, nhóm mong muốn:

- Áp dụng kiến thức hợp ngữ MIPS để giải quyết một bài toán thực tế, tăng khả năng tư duy thuật toán và tổ chức chương trình ở mức thấp.
- Làm quen với tư duy thiết kế phần mềm hệ thống có cấu trúc mô-đun rõ ràng, dễ bảo trì.
- Cải thiện kỹ năng debug, kiểm soát logic, và quản lý dòng chảy chương trình một cách chặt chẽ trong môi trường giới hạn tài nguyên.
- Tăng tính sáng tạo và chủ động bằng cách phát triển thêm các tính năng nâng cao như AI, undo, custom board, timeout,...

Các mục tiêu cụ thể đã được nhóm triển khai gồm:

- Hiện thực chế độ chơi với AI: người chơi có thể lựa chọn đấu với máy, trong đó AI được lập trình để đưa ra nước đi hợp lệ dựa trên thời gian giới hạn cho mỗi lượt. Điều này không chỉ tăng tính thực tế mà còn giúp kiểm tra khả năng xử lý logic và chuyển lượt tự động của chương trình.
- Giao diện văn bản thân thiện với người dùng trong terminal, dễ quan sát với bảng cờ căn chỉnh rõ ràng, tên người chơi và tọa độ trực quan.
- Thiết kế hệ thống macro riêng biệt giúp tổ chức lại toàn bộ chương trình, dễ tái sử dụng và bảo trì.
- Kiểm tra định dạng và giới hạn tọa độ người dùng nhập vào, đảm bảo tính hợp lệ và thân thiện khi thao tác.
- Ghi kết quả sau mỗi ván đấu ra tệp result.txt để hỗ trợ lưu trữ và tra cứu.
- Kiểm tra điều kiện thắng theo 4 hướng: ngang, dọc, chéo xuôi, chéo ngược.
- Hỗ trợ đầu hàng, undo, và giới hạn thời gian chơi.
- Chế độ custom cho phép người dùng tự thiết lập kích thước bàn cờ, điều kiện thắng và giới hạn thời gian.



- Quản lý biến trạng thái bằng thanh ghi \$s, kiểm soát luồng chương trình chính xác, hạn chế lỗi logic.

Thông qua đề tài, nhóm rèn luyện kỹ năng sử dụng lệnh hợp ngữ MIPS (li, la, syscall, move, beq, bne, lw, sw,...) và tư duy thiết kế phần mềm hệ thống. Đây là nền tảng quan trọng cho các dự án nhúng và hệ thống sau này.

3 Phương Pháp Thực Hiện Đề Tài

Để đảm bảo tính rõ ràng và mở rộng, chương trình được modular hóa thông qua các file:

- **AI_part.mac**: Thuật toán logic giúp máy (AI) xử lý nước đi trong giới hạn thời gian.
- **Console_part.mac**: In bảng cờ, ASCII art và giao diện người dùng.
- **Data_part.mac**: Khai báo biến toàn cục, trạng thái và cấu trúc dữ liệu.
- **File_part.mac**: Ghi trạng thái bảng cờ và kết quả ván chơi ra file.

Logic Hoạt Động Của Chương Trình

1. Hiện thị logo ASCII và yêu cầu chọn chế độ chơi.
2. Nếu chọn custom, nhập thông số: kích thước bàn, số quân thắng, thời gian.
3. Reset bảng cờ, khởi tạo lượt chơi, đặt biến trạng thái ban đầu.
4. Vòng lặp trò chơi chính:
 - Kiểm tra timeout của người chơi hiện tại.
 - Hiện thị bảng, lấy input (tọa độ, undo, đầu hàng).
 - Thực hiện nước đi nếu hợp lệ, cập nhật lượt và kiểm tra kết thúc.
 - Nếu chưa kết thúc, chuyển lượt và tiếp tục.
5. Nếu kết thúc: hiện thị kết quả, lưu trạng thái, hỏi có chơi lại không.
6. Người chơi chọn: tiếp tục, chọn chế độ khác, hoặc thoát.

Các Chức Năng Nâng Cao Và Cải Tiến

- **Undo, đầu hàng**: Có thể hoàn tác nước đi gần nhất của người chơi, logic tự động chuyển lượt hợp lý, có thể đầu hàng và kết thúc trận đấu ngay lập tức.
- **Timeout chính xác**: Mỗi lượt đều kiểm tra thời gian thực, xử lý tình huống vượt thời gian hợp lý.
- **Lưu trạng thái trận đấu**: Bao gồm bảng cờ, thông báo kết thúc, giúp giảng viên kiểm tra dễ dàng.
- **Replay thông minh**: Tự động chơi lại với cấu hình cũ hoặc tùy chọn thay đổi chế độ.
- **Lựa chọn chế độ linh hoạt**: Có nhiều chế độ cho người chơi chọn lựa, các thông số như kích thước bàn, thời gian,... không cố định mà có thể thay đổi. Có các chế độ chơi với máy (AI), chế độ cờ thế (puzzle),...

Phân Tích Kỹ Thuật Và Tính Ứng Dụng

- Tối ưu bằng thanh ghi: hạn chế truy cập bộ nhớ, giảm độ trễ xử lý.
- Kiểm soát luồng chương trình rõ ràng, dễ debug và mở rộng.
- Nhập liệu được kiểm tra định dạng chặt chẽ, tăng trải nghiệm người dùng.
- Cân bằng giữa hiệu năng và tính năng, phù hợp chạy trong môi trường MARS giả lập.
- Có tính mở rộng cao: Các thông số không cố định giúp người chơi có nhiều lựa chọn, đồng thời dễ có những ý tưởng cải tiến game hơn.

4 Giải Thuật Chính Của Chương Trình

Trong bài tập lớn lần này, nhóm chú trọng vào macro check dir và check win như là giải thuật trọng tâm và cốt lõi của bài toán trò chơi Gomoku, cụ thể:

Hàm check_dir

Mục đích Đếm số quân liên kề cùng màu của người chơi theo một hướng xác định và hướng ngược lại, bắt đầu từ ô vừa đặt.

Đầu vào

- Tọa độ ô vừa đặt: (x, y)
- Vector hướng kiểm tra: (dx, dy)
- Mã người chơi: $player \in \{1, 2\}$

Đầu ra Số quân liên tiếp tính cả ô gốc: $count \in \mathbb{N}$

1. Khởi tạo biến đếm:

$$count \leftarrow 1$$

2. Quét theo hướng tiến (dx, dy) :

(a) Lặp lại:

$$x \leftarrow x + dx, \quad y \leftarrow y + dy$$

(b) Nếu (x, y) nằm ngoài bàn hoặc ô (x, y) không chứa quân của $player$, dừng.

(c) Ngược lại, $count \leftarrow count + 1$ và tiếp tục lặp.

3. Quay về tọa độ gốc.

4. Quét theo hướng ngược $(-dx, -dy)$ tương tự bước 2, mỗi lần trùng màu tăng $count$.

5. Kết quả trả về: $count$.

Hàm check_win

Mục đích Kiểm tra xem nước đi mới nhất có tạo thành chuỗi 5 quân thắng cuộc theo bất kỳ hướng ngang, dọc, chéo chính, hoặc chéo phụ hay không.

Đầu vào $(x, y), player$

Đầu ra Giá trị logic: true nếu thắng, ngược lại false.

1. Khởi $win \leftarrow false$.

2. Lần lượt gọi `check_dir` với bốn hướng:

$$(dx, dy) \in \{(1, 0), (0, 1), (1, 1), (1, -1)\}.$$

3. Nếu bất kỳ lần gọi nào trả về $count \geq 5$, gán $win \leftarrow \text{true}$ và dừng kiểm tra.

4. Trả về win .

Vector hướng

Khái niệm Vector (dx, dy) xác định cách “nhảy” từ ô hiện tại (x, y) đến ô kế tiếp trên bàn cờ:

$$(x', y') = (x + dx, y + dy).$$

Ý nghĩa trên bàn Gomoku 15×15 : • **Ngang (sang phải):** $(1, 0)$ và ngược lại $(-1, 0)$.

- **Dọc (xuống dưới):** $(0, 1)$ và ngược lại $(0, -1)$.
- **Chéo chính (xuống phải):** $(1, 1)$ và ngược lại $(-1, -1)$.
- **Chéo phụ (lên phải):** $(1, -1)$ và ngược lại $(-1, 1)$.

Chức năng Giúp hàm `check_dir` xác định chính xác đường thẳng cần đếm quân liên tiếp.

Ví dụ minh họa

Giả sử trên bàn 15×15 , người chơi 1 vừa đặt quân tại ô $(7, 7)$. Trước và sau ô này trên hàng $y = 7$ đã có quân 1 tại các ô:

$$(5, 7), (6, 7) \quad \text{và} \quad (8, 7), (9, 7).$$

- Gọi `check_dir(7,7;1,0;1)` (hướng ngang):
 - Quét tiến: từ $(7, 7) \rightarrow (8, 7) \rightarrow (9, 7)$ đều trùng màu $\Rightarrow$ cộng 2.
 - Quét lùi: từ $(7, 7) \rightarrow (6, 7) \rightarrow (5, 7)$ đều trùng màu $\Rightarrow$ cộng 2.
 - Tổng $count = 1 + 2 + 2 = 5$.
- Trong `check_win`, vì $count \geq 5$ ở hướng ngang, kết luận *thắng* và không kiểm tra thêm.

5 Hiện Thực Các Hàm Bổ Trợ (Macro)

1. In logo trò chơi (display_label)

- **Tên hàm:** display_label.
- **Chức năng:** Hiện thị logo ASCII "GOMOKU" được lưu trong 6 mảng chuỗi gomoku_art một cách nghệ thuật trước khi trò chơi bắt đầu.
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** Không.
- **Thanh ghi sử dụng:** \$a0, \$v0.

2. Chọn chế độ chơi (choose_mode)

- **Tên hàm:** choose_mode.
- **Chức năng:** Hỏi xem người chơi muốn chơi ở chế độ nào:
 - Default mode(d): Như mô tả trong Bài tập lớn, bàn cờ kích thước 15x15, 5 quân liên tiếp theo một trong 3 hướng dọc, ngang, chéo để chiến thắng. Ngoài ra, giới hạn thời gian tối đa ở 600s, không cộng thêm.
 - Custom mode(c): Tùy chỉnh kích thước bàn cờ (1-100), số lượng ô liên tiếp để chiến thắng (1-10), giới hạn thời gian (1-1000 hoặc không giới hạn), cộng thêm thời gian (0-10).
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** \$v0 (0 nếu default mode, 1 nếu custom mode).
- **Tác động đến biến toàn cục:** SIZE, NUM_TO_WIN, TIME_LIMIT, game_mode.
- **Thanh ghi sử dụng:** \$t0-\$t2, \$t9, \$s1-\$s2, \$s5, \$a0-\$a1, \$v0

3. Chọn kiểu đối thủ (choose_opponent)

- **Tên hàm:** choose_opponent.
- **Chức năng:** Hỏi xem người chơi muốn chơi với kiểu đối thủ nào:
 - Human(h): Như mô tả trong Bài tập lớn, hai người chơi lần lượt đưa ra các nước đi của mình cho đến khi một trong hai chiến thắng hoặc không còn nước nào để đi (hòa).
 - AI(a): Người chơi chọn một trong hai loại quân cờ (X hoặc O), quân X sẽ được đi trước, một giải thuật đã được lập trình trước để quyết định nước đi tối ưu cho AI, người chơi cần tìm cách để chiến thắng AI.
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** game_opponent (0 nếu người đấu với người, 1 nếu người đấu với AI).
- **Thanh ghi sử dụng:** \$t0-\$t1, \$a0-\$a1, \$v0

4. Chọn chế độ puzzle (ask_puzzle_type)

- **Tên hàm:** ask_puzzle_type.
- **Chức năng:** (Chỉ áp dụng trong chế độ custom) Hỏi xem người chơi có muốn chọn chế độ blank hay puzzle:
 - Blank(b): Bàn cờ trống, không có chướng ngại vật hay nước đi nào được thực hiện trước khi trò chơi bắt đầu.
 - Puzzle(p): Bàn cờ có sẵn một vài ô được đánh X, O một cách ngẫu nhiên và công bằng theo công thức $\frac{n*n}{16}$, đồng thời đảm bảo số nước có sẵn không nhỏ hơn 2 và là số chẵn.
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** \$v0 (0 nếu không đồng ý, 1 nếu đồng ý).
- **Tác động đến biến toàn cục:** Không.
- **Thanh ghi sử dụng:** \$t0-\$t1, \$a0-\$a1, \$v0

5. Tạo các nước đi ngẫu nhiên (generate_random_moves)

- **Tên hàm:** generate_random_moves.
- **Chức năng:** (Dùng trong chế độ puzzle) Tạo ngẫu nhiên SIZE*SIZE/16 nước đi chia đều cho hai bên (làm tròn lên số chẵn gần nhất).
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** board, random_count.
- **Thanh ghi sử dụng:** \$t0-\$t8, \$a0-\$a1, \$v0

6. Chọn thông tin cho trò chơi (choose_game_information)

- **Tên hàm:** choose_game_information.
- **Chức năng:** Các thiết lập cơ bản trước khi bắt đầu ván chơi, bao gồm chế độ chơi, loại câu đố (nếu có), và lựa chọn đối thủ. Hàm này viết ra nhằm tổng hợp 3 hàm khác bao gồm choose_mode, ask_puzzle_type và choose_opponent nhằm rút ngắn độ dài hàm Main.
 - Gọi macro choose_mode để chọn chế độ chơi (chơi thường hoặc puzzle).
 - Nếu người chơi chọn chế độ puzzle, gọi macro ask_puzzle_type để xác định loại câu đố và lưu vào biến toàn cục puzzle_type.
 - Gọi macro choose_opponent để xác định đối thủ là người hay AI.
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Lưu giá trị lựa chọn vào biến toàn cục puzzle_type và thanh ghi \$v0.
- **Tác động đến biến toàn cục:** puzzle_type.
- **Thanh ghi sử dụng:** \$v0.

7. Tạo bản sao lưu của bàn cờ ban đầu (create_initial_board)

- **Tên hàm:** create_initial_board.
- **Chức năng:** Sao chép trạng thái hiện tại của bàn cờ (board) vào một bản sao gọi là init_board, nhằm lưu giữ lại tình trạng ban đầu của ván cờ (ngay sau khởi tạo hoặc sinh nước đi puzzle).
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** init_board.
- **Thanh ghi sử dụng:** \$t0-\$t6.

8. Khởi động bộ đếm thời gian (start_timer)

- **Tên hàm:** start_timer.
- **Chức năng:** Gán giới hạn thời gian vào bộ đếm của 2 người chơi, bắt đầu đếm thời gian bằng syscall 30, gán thời gian lúc bắt đầu đếm vào biến last_tick.
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** player1_time, player2_time, last_tick.
- **Thanh ghi sử dụng:** \$s5, \$a0, \$v0.

9. Cập nhật thời gian (update_player_time)

- **Tên hàm:** update_player_time.
- **Chức năng:** Cập nhật thời gian còn lại của người chơi hiện tại thông qua biến last_tick (thời điểm bắt đầu đếm).
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.

- **Tác động đến biến toàn cục:** player1_time, player2_time, last_tick.
- **Thanh ghi sử dụng:** \$t0-\$t2, \$t9, \$s0, \$a0, \$v0.

10. Cập nhật thời gian còn lại của AI (update_ai_time)

- **Tên macro:** update_ai_time.
- **Chức năng:** Giảm thời gian còn lại của AI sau mỗi lượt đi, trừ khi đang ở chế độ không giới hạn thời gian.
 - Đầu tiên, kiểm tra biến toàn cục TIME_LIMIT. Nếu giá trị này âm (tức là không giới hạn thời gian), AI sẽ không bị trừ thời gian và thoát macro.
 - Nếu có giới hạn thời gian, macro sẽ xác định AI đang chơi ở vai trò người chơi 1 hay người chơi 2 thông qua biến human_side.
 - Giảm 5 giây khỏi thời gian tương ứng của người chơi AI (player1_time hoặc player2_time).
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** TIME_LIMIT, player1_time, player2_time, human_side.
- **Thanh ghi sử dụng:** \$t2, \$t4.

11. Cộng thêm thời gian cho người chơi sau mỗi nước đi (add_player_time)

- **Tên hàm:** add_player_time.
- **Chức năng:** Thêm thời gian (ADD_TIME) vào bộ đếm của người chơi hiện tại sau mỗi nước đi hợp lệ (chỉ trong chế độ có thời gian cộng thêm).
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** player1_time, player2_time, ADD_TIME, current_player.
- **Thanh ghi sử dụng:** \$t0-\$t1, \$s0.

12. Xác định bên người chơi cần kiểm tra (determine_side_check)

- **Tên macro:** determine_side_check.
- **Chức năng:** Xác định bên người chơi hiện tại (human hoặc AI) để quyết định nên gọi macro kiểm tra thời gian tương ứng nào.
 - Đầu tiên, macro tải giá trị từ biến human_side và game_opponent.
 - Nếu không ở chế độ đấu với AI (game_opponent = 0), thì kiểm tra thời gian của người chơi thông thường bằng nhãn human_check.
 - Nếu đang ở chế độ đấu AI:
 - * Nếu human_side = 1 (người là player 1) và current_player = 2 → lượt của AI, nhảy đến ai_check.
 - * Nếu human_side = 2 (người là player 2) và current_player = 1 → lượt của AI, nhảy đến ai_check.
 - Nếu không phải lượt của AI, quay lại kiểm tra như thông thường.
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** human_side, game_opponent, current_player.
- **Thanh ghi sử dụng:** \$t3-\$t4, \$s0.

13. Kiểm tra hết giờ (check_timeout)

- **Tên hàm:** check_timeout.
- **Chức năng:** Kiểm tra xem người chơi hiện tại đã hết thời gian chưa. Nếu trong chế độ không giới hạn thời gian thì bỏ qua
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** \$v0 (1 nếu hết thời gian, 0 nếu còn thời gian chơi).
- **Tác động đến biến toàn cục:** Không.
- **Thanh ghi sử dụng:** \$t9, \$a0, \$v0.

14. Kiểm tra AI hết thời gian (check_ai_timeout)

- **Tên macro:** check_ai_timeout.
- **Chức năng:** Kiểm tra xem AI có hết thời gian giới hạn cho lượt đi hiện tại hay không.
 - Khởi tạo $\$v0 = 0$ — giả định chưa hết thời gian.
 - Xác định xem AI là người chơi thứ nhất hay thứ hai dựa vào biến human_side.
 - Tải giá trị TIME_LIMIT, nếu là giá trị âm (tức là chế độ không giới hạn thời gian), bỏ qua kiểm tra timeout.
 - Dựa vào vai trò của AI:
 - * Nếu là người chơi thứ nhất, kiểm tra player1_time.
 - * Nếu là người chơi thứ hai, kiểm tra player2_time.
 - Nếu thời gian còn lại nhỏ hơn 0, gán $\$v0 = 1$ để báo hiệu timeout.
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** $\$v0 = 1$ nếu AI đã hết thời gian; ngược lại $\$v0 = 0$.
- **Tác động đến biến toàn cục:** TIME_LIMIT, player1_time, player2_time, human_side.
- **Thanh ghi sử dụng:** \$t2, \$t4, \$v0.

15. Đặt thời gian về 0 khi hết giờ (set_time_to_zero)

- **Tên hàm:** set_time_to_zero.
- **Chức năng:** Đặt thời gian còn lại của người chơi hiện tại về 0 khi hết giờ để tránh hiển thị số âm.
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** player1_time, player2_time, current_player.
- **Thanh ghi sử dụng:** \$t0, \$s0.

16. Hiển thị thời gian còn lại của người chơi (display_time_remaining)

- **Tên hàm:** display_time_remaining.
- **Chức năng:** Hiển thị thời gian còn lại của cả hai người chơi ra màn hình console.
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** player1_time, player2_time, TIME_LIMIT.
- **Thanh ghi sử dụng:** \$t0-\$t1, \$a0-\$a1, \$v0.

17. Hiển thị số nước cờ 2 người chơi đã đi (display_move_count)

- **Tên hàm:** display_move_count.
- **Chức năng:** Hiển thị số nước đi hiện tại.
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** Không.
- **Thanh ghi sử dụng:** \$s7, \$a0, \$v0.

18. Hỏi chơi lại (ask_replay)

- **Tên hàm:** ask_replay.
- **Chức năng:** Hỏi người chơi có muốn chơi lại chế độ hiện tại không hay là muốn đổi chế độ.
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** \$v0 (1 nếu người chơi muốn chơi lại, 0 nếu có và 2 nếu muốn đổi chế độ chơi).
- **Tác động đến biến toàn cục:** Không.
- **Thanh ghi sử dụng:** \$t0-\$t1, \$a0-\$a1, \$v0

19. Đặt lại ván mới (reset_game)

- **Tên hàm:** reset_game.
- **Chức năng:** Hỏi người chơi có muốn chơi mới lại từ đầu không.
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** player1_time, player2_time, last_tick, board, init_board, current_player, move_count, undo_flag, action_count, win_start_x, win_start_y, win_end_x, win_end_y.
- **Thanh ghi sử dụng:** \$t0-\$t3, \$s0, \$s7, \$a0, \$v0

20. In bàn cờ thu nhỏ (print_mini_board)

- **Tên hàm:** print_mini_board.
- **Chức năng:** In phiên bản thu nhỏ 15x15 của bàn cờ xung quanh nước đi gần nhất khi kích thước lớn hơn 20x20 để dễ quan sát.
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** board.
- **Thanh ghi sử dụng:** \$t0-\$t7, \$s1, \$a0, \$a3, \$v0.

21. In ra bàn cờ hiện tại (print_board)

- **Tên hàm:** print_board.
- **Chức năng:** In ra màn hình tình trạng hiện tại của bàn cờ, bao gồm những nước cờ đã đi.
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.

- **Tác động đến biến toàn cục:** Không.
- **Thanh ghi sử dụng:** \$t0-\$t7, \$s1, \$a0, \$a3, \$v0

22. Xử lý đầu vào của người chơi (get_player_input)

- **Tên hàm:** get_player_input.
- **Chức năng:** Nhận đầu vào từ người chơi, xử lý chuỗi nhập vào từ console, và chuyển đổi nó thành tọa độ nước đi (x, y) hoặc một lệnh đặc biệt như undo hoặc surrender.
 - Nếu chuỗi là dạng "x, y": Phân tách chuỗi, lấy phần trước dấu , là x, sau là y. Chuyển chuỗi số thành số nguyên, trả về: \$v0 = x, \$v1 = y
 - Nếu chuỗi là "u": Trả về: \$v0 = -1 (undo)
 - Nếu chuỗi là "s": Trả về: \$v0 = -2 (surrender)
 - Nếu chuỗi không hợp lệ: In ra một trong các lỗi theo tùy điều kiện : "Invalid command! Enter as x,y or 'u' or 's'.", "Input out of range! Please try again.", "Invalid input! Please enter only digits." và yêu cầu nhập lại.
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** \$v0 = x coordinate, \$v1 = y coordinate, hoặc \$v0 = -1 (undo), \$v0 = -2 (surrender).
- **Tác động đến biến toàn cục:** Không.
- **Thanh ghi sử dụng:** \$t0-\$t9, \$s1, \$s7, \$a0, \$v0-\$v1

23. Thực hiện di chuyển (make_move)

- **Tên hàm:** make_move.
- **Chức năng:** Đặt quân cờ vào vị trí (x, y) trên bàn cờ nếu chưa có ai chiếm, đồng thời cập nhật trạng thái nước đi cuối cùng.
 - Hàm nhận vào ba tham số: tọa độ x, y, và player (1 hoặc 2), tức người chơi muốn đánh quân tại vị trí (x, y). Đầu tiên, nó lưu lại thông tin nước đi này bằng cách: Ghi x vào biến toàn cục last_move_x, ghi y vào last_move_y, ghi người chơi (1 hoặc 2) vào last_player. Việc này giúp ghi nhớ nước đi cuối cùng, dùng để: hiển thị ra màn hình, kiểm tra thắng, ghi vào lịch sử, xử lý undo.

- Sau đó, hàm tính vị trí tương ứng trong mảng bàn cờ một chiều: Nếu ô còn trống (giá trị là 0), thì: ghi người chơi (1 hoặc 2) vào ô đó, đặt $\$v0 = 0$ để báo rằng nước đi thành công. Nếu ô đã bị chiếm (occupied): Tải địa chỉ chuỗi thông báo lỗi "Position already occupied! Try again." vào $\$t0$, ghi thông báo đó vào `current_message` (dùng để hiển thị sau khi in bàn) và đặt $\$v0 = 1$ để báo lỗi (ô đã có quân).

- **Vị trí:** `Console_part.mac`.
- **Tham số:** $\$x$, $\$y$, $\$player$ (1 hoặc 2).
- **Giá trị trả về:** $\$v0$ (1 nếu ô đã bị được đánh rồi, 0 nếu ô đó trống có thể đánh)
- **Tác động đến biến toàn cục:** `last_move_x`, `last_move_y`, `last_player`, `current_message`, `board`.
- **Thanh ghi sử dụng:** $\$t2$ - $\$t6$, $\$v0$, $\$x$, $\$y$, $\$player$

24. Thực hiện nước đi của AI (`ai_make_move`)

- **Tên hàm:** `ai_make_move`.
- **Chức năng:** Tự động chọn nước đi cho AI dựa trên một chiến lược phù hợp mà nhóm đã nghĩ ra. Đồng thời trong quá trình tìm nước đi tối ưu ở từng ưu tiên, nhóm cũng tìm cách loại bỏ các nước đi không cần thiết (vô nghĩa) để tránh lãng phí nước đi. Đầu tiên, duyệt toàn bộ các ô trên bàn cờ (`board`) bằng hai vòng lặp lồng nhau, bỏ qua các ô đã có người chơi (giá trị khác 0), với mỗi ô trống, AI thực hiện kiểm tra theo thứ tự ưu tiên như sau:
 - **Ưu tiên 1 (Priority 1) – Đánh để thắng ngay:**
 - * AI duyệt qua từng ô trống trên bàn cờ và giả lập việc đặt quân của mình vào vị trí đó bằng cách gán giá trị `board[y][x] ← current_player`.
 - * Sau đó, gọi macro `check_win(x, y, current_player)` để kiểm tra xem nếu thực sự thực hiện nước đi đó, AI có thể thắng ngay lập tức hay không.
 - * Nếu macro `check_win` trả về kết quả thắng:
 - Gán $\$v0 ← x$, $\$v1 ← y$ để lưu tọa độ nước đi tối ưu.
 - Xóa quân vừa giả lập để khôi phục trạng thái bàn cờ.
 - Kết thúc macro ngay tại đây vì đây là nước đi tốt nhất — AI đã có thể chiến thắng, không cần xét thêm các khả năng khác.
 - **Ưu tiên 2 (Priority 2) – Chặn nước thắng của đối thủ:**
 - * AI quét toàn bộ bàn cờ, kiểm tra theo 8 hướng từ mỗi ô và giả lập từng chuỗi 5 ô liên tiếp có thể chứa 4 quân của người chơi human và 1 ô trống.

- * Với mỗi chuỗi như vậy, nếu có đúng 4 quân của human và chỉ duy nhất 1 ô trống, AI xác định đó là nước đi cần chặn gấp.
 - * Khi tìm được tọa độ ô cần chặn ($empty_x$, $empty_y$), AI gán giá trị $\$v0 \leftarrow empty_x$, $\$v1 \leftarrow empty_y$, và kết thúc macro — vì đây là nước đi bắt buộc để ngăn đối thủ thắng ngay sau đó.
- **Ưu tiên 3 (Priority 3) – Tạo chuỗi 4 từ 3 quân của AI:**
- * AI duyệt các chuỗi 4 ô liên tiếp theo 8 hướng từ mỗi vị trí trên bàn cờ, với mục tiêu tìm các chuỗi có đúng 3 quân AI và 1 ô trống.
 - * Nếu tìm thấy chuỗi như vậy, AI tiếp tục kiểm tra hai đầu của chuỗi (đầu và đuôi) xem có ít nhất một đầu còn trống hay không để bảo đảm đây là chuỗi “mở” và có tiềm năng phát triển thành 5 quân liên tiếp.
 - * Nếu chuỗi đủ điều kiện (3 quân AI + 1 ô trống + ít nhất 1 đầu chuỗi mở):
 - Gán $\$v0 \leftarrow x$, $\$v1 \leftarrow y$ với tọa độ của ô trống.
 - Kết thúc macro để thực hiện nước đi — vì đây là bước tiến quan trọng giúp AI tiến gần đến chiến thắng.
- **Ưu tiên 4 (Priority 4) – Chặn chuỗi 3 quân của đối thủ:**
- * AI quét toàn bộ bàn cờ, duyệt theo 8 hướng để tìm các chuỗi 4 ô liên tiếp có chứa đúng 3 quân của người chơi human và 1 ô trống, không có quân AI xen vào.
 - * Với mỗi chuỗi hợp lệ, AI kiểm tra thêm hai đầu chuỗi (đầu trước và đầu sau) có trống không, nhằm đảm bảo đây là chuỗi “mở hai đầu” — nguy hiểm và cần chặn sớm.
 - * Nếu cả hai đầu đều trống, AI xác định đây là một chuỗi đe dọa tiềm năng cần phải ngăn chặn ngay.
 - Gán $\$v0 \leftarrow x$, $\$v1 \leftarrow y$ với tọa độ ô trống trong chuỗi.
 - Kết thúc macro để AI thực hiện nước đi chặn — giúp phá vỡ chiến lược tấn công của đối phương trước khi nó trở nên nguy hiểm hơn.
- **Ưu tiên 5 (Priority 5) – Xây dựng tại giao điểm của hai đường tấn công tiềm năng của AI:**
- * AI duyệt toàn bộ các ô trống trên bàn cờ để tìm những vị trí chiến lược có thể là giao điểm của hai đường tiềm năng (hai hướng khác nhau) mà mỗi đường đều chứa ít nhất 2 quân AI.
 - * Với mỗi ô trống (x, y) :
 - Duyệt qua từng cặp hướng khác nhau (từ 0 đến 7, trừ các cặp ngược nhau).
 - Với mỗi hướng, kiểm tra theo cả hai chiều (âm và dương), cho phép tối đa 1 ô trống (gap) nằm giữa các quân AI.

- Tổng số quân AI liên tiếp (có thể cách nhau 1 ô trống) ở mỗi hướng phải đạt ít nhất 2.
- * Sau khi tìm được 2 hướng thỏa mãn, AI kiểm tra xem cả hai hướng có thực sự “có ý nghĩa” chiến lược không:
 - Một hướng được xem là “có ý nghĩa” nếu tồn tại ít nhất một trong ba cặp vị trí đầu/cuối trống không bị chặn bởi quân người chơi:
 - Các cặp cần xét: $(-1, 3)$, $(-2, 2)$, $(-3, 1)$ tính theo vectơ hướng.
 - Nếu ít nhất một vị trí trong mỗi cặp là ô trống hoặc không bị chặn, hướng đó được xem là tiềm năng để mở rộng.
 - Chỉ cần một trong hai hướng là “có ý nghĩa” là đủ để xem vị trí (x, y) là giao điểm tốt.
- * Nếu một ô thỏa mãn các điều kiện trên:
 - Gán $\$v0 \leftarrow x$, $\$v1 \leftarrow y$ để lưu tọa độ nước đi tiềm năng.
 - In chuỗi ký hiệu tương ứng (strP5) ra màn hình để đánh dấu lý do chọn.
 - Kết thúc macro ngay tại đây — nước đi đã được quyết định.
- **Ưu tiên 6 (Priority 6) – Chặn giao điểm 2 đường tiềm năng của người:**
 - * AI duyệt từng ô trống trên bàn và kiểm tra xem ô đó có nằm trên ít nhất 2 hướng khác nhau mà mỗi hướng có từ 2 quân người, được ngắt quãng bởi tối đa 1 ô trống.
 - * Với mỗi cặp hướng, nếu cả hai đều được xem là “có tiềm năng” (hai đầu còn mở và không bị AI chặn):
 - Gán tọa độ ô đang xét vào $\$v0$, $\$v1$ làm nước đi phòng thủ tối ưu.
 - In thông báo strP6 và kết thúc macro.
- **Ưu tiên 7 (Priority 7) – Xây chuỗi 3 từ 2 quân AI:**
 - * AI duyệt các hướng từ mỗi ô trên bàn để tìm chuỗi 3 ô liên tiếp chứa 2 quân AI và 1 ô trống, không chứa quân người.
 - * Nếu tìm được một chuỗi hợp lệ:
 - Xác định ô trống trong chuỗi và kiểm tra xem có ít nhất một đầu (head hoặc tail) của chuỗi đó không bị chặn (trống).
 - Nếu có, gán tọa độ ô trống đó vào $\$v0$, $\$v1$ làm nước đi tiếp theo.
 - In thông báo strP7 và kết thúc macro.
- **Ưu tiên 8 (Priority 8) – Chặn 2 quân liên tiếp của người:**
 - * Duyệt từng ô trống và từng hướng (0–7). Xác định chuỗi 3 ô liên tiếp chứa chính xác 2 quân người và 1 ô trống.

- * Đảm bảo cả hai đầu của chuỗi (head và tail) đều trống, để nước chặn trở nên có ý nghĩa.
- * Nếu thỏa điều kiện, lưu tọa độ ô trống vào \$v0, \$v1, in thông báo strP8, rồi kết thúc.

– **Ưu tiên 9 (Priority 9) – Xây chuỗi 2 từ 1 quân AI:**

- * Duyệt từng ô trống theo từng hướng (0–7), tìm chuỗi 2 ô liền kề chứa đúng 1 quân AI và 1 ô trống.
- * Đảm bảo ít nhất một đầu của chuỗi (head hoặc tail) là ô trống, giúp chuỗi có khả năng mở rộng.
- * Nếu thỏa điều kiện, lưu tọa độ ô trống vào \$v0, \$v1, in thông báo strP9, rồi kết thúc.

– **Ưu tiên 10 (Priority 10) – Đánh gần cả quân người chơi và quân AI:**

- * AI bắt đầu bằng cách lấy tọa độ nước đi gần nhất của người chơi từ hai biến last_move_x và last_move_y.
- * Từ vị trí này, AI quét qua 8 hướng lân cận (tương ứng với 8 hướng trên bàn cờ) và kiểm tra từng ô kề cận xem ô đó có trống không.
- * Nếu ô trống được phát hiện:
 - AI tiếp tục kiểm tra các ô xung quanh ô trống đó (cũng theo 8 hướng) để xem có tồn tại quân AI gần kề hay không.
 - Nếu có ít nhất một quân AI cạnh ô trống, AI đánh giá đây là một nước đi "tối ưu" vì vừa áp sát người chơi, vừa duy trì thế liên kết với quân AI hiện có.
- * Nếu tìm thấy ô trống thỏa mãn điều kiện trên:
 - In chuỗi thông báo strP10, báo rằng đây là nước đi theo **Priority 10**.
 - Gán \$v0 ← x, \$v1 ← y là tọa độ ô trống đó.
 - Kết thúc macro để thực hiện nước đi.
- * Nếu không tìm thấy ô phù hợp (gần cả người và AI), AI chuyển sang ưu tiên tiếp theo (Priority 11).

– **Ưu tiên 11 (Priority 11) – Đánh gần quân người chơi:**

- * Nếu không tìm được nước đi nào thỏa mãn điều kiện của Priority 10 (gần cả quân người chơi và AI), AI sẽ chuyển sang kiểm tra các ô trống **gần quân người chơi** thôi.
- * Dựa trên tọa độ nước đi gần nhất của người chơi (last_move_x, last_move_y), AI tiếp tục quét 8 hướng xung quanh vị trí đó.

- * Với mỗi ô trống được phát hiện:
 - AI lưu lại vị trí đầu tiên thoả mãn là "ứng viên" tiềm năng cho nước đi.
 - Tuy không có quân AI gần đó (không đủ điều kiện Priority 10), nhưng vẫn có giá trị vì giúp duy trì áp lực lên người chơi.
- * Nếu đến cuối vòng lặp mà không có ô nào gần AI nhưng có ô gần người, AI sẽ:
 - In chuỗi thông báo strP11, báo rằng đây là nước đi theo **Priority 11**.
 - Gán $\$v0 \leftarrow x$, $\$v1 \leftarrow y$ với tọa độ ô trống gần người chơi.
 - Kết thúc macro để thực hiện nước đi.
- * Nếu không tìm thấy ô nào phù hợp, AI sẽ chuyển sang Priority 12 để đánh gần một nước người chơi khác (ngẫu nhiên hơn).

– **Ưu tiên 12 (Priority 12) – Đánh gần quân người chơi bất kỳ:**

- * Nếu AI không tìm được nước đi nào gần người chơi và AI (Priority 10) hoặc chỉ gần người chơi gần nhất (Priority 11), nó sẽ mở rộng tìm kiếm.
- * Đầu tiên, AI sẽ đếm tổng số nước đi của người chơi hiện có trên bàn cờ (các ô có giá trị human_side).
- * Sau đó, chọn ngẫu nhiên một trong các vị trí đó và sử dụng nó làm tâm để kiểm tra các ô trống xung quanh.
- * Với mỗi vị trí mới chọn, thực hiện lại ưu tiên 10 và 11 đến khi tìm được nước đi phù hợp
- * Khi xác định được nước đi:
 - AI in thông báo strP12 để biểu thị rằng đây là nước đi ưu tiên thứ 12.
 - Gán $\$v0 \leftarrow x$, $\$v1 \leftarrow y$ là tọa độ ô trống vừa chọn.
 - Kết thúc macro và thực hiện nước đi.
- * Nếu vẫn không tìm được nước đi khả thi, AI sẽ chuyển sang chế độ đánh hoàn toàn ngẫu nhiên trên toàn bàn cờ.

– **Ưu tiên 13 (Priority 13) – Đánh gần một ô đã có bất kỳ (± 2 ô):**

- * AI sẽ quét toàn bộ bàn cờ để đếm tổng số ô đã được đánh (bao gồm cả của người chơi và AI).
- * Nếu không có nước đi nào đã được đánh trước đó, AI chuyển sang đánh ngẫu nhiên.
- * Nếu có, AI chọn ngẫu nhiên một ô trong số đó làm "tâm".
- * Từ ô tâm này, AI duyệt tất cả các ô nằm trong phạm vi ± 2 theo cả hai chiều (x và y) — tạo thành một khung 5x5.

- Nếu ô đang xét trống và không phải là chính giữa (tâm), AI chọn ô đó là nước đi.
- Khi tìm được ô trống hợp lệ, AI gán tọa độ vào $\$v0 \leftarrow x$ và $\$v1 \leftarrow y$, rồi kết thúc macro.
- AI cũng in ra chuỗi strP13 để xác nhận đây là nước đi theo ưu tiên 13.
- * Nếu sau khi duyệt toàn bộ vùng lân cận vẫn không tìm được ô trống, AI sẽ chuyển sang chiến lược đánh ngẫu nhiên hoàn toàn (fallback).

– **Ưu tiên 14 (Priority 14) – Nước đi hoàn toàn ngẫu nhiên:**

- * Đây là chiến lược cuối cùng khi tất cả các ưu tiên khác đều không tìm được nước đi phù hợp.
- * AI thực hiện lựa chọn ngẫu nhiên tọa độ (x, y) trong phạm vi kích thước bàn cờ.
- * Với mỗi cặp (x, y) được tạo ra:
 - AI kiểm tra ô tương ứng trên bảng. Nếu ô đó trống, nó được chọn làm nước đi.
 - Nếu ô không trống (đã có quân cờ), AI tiếp tục lặp lại quy trình để tạo ra tọa độ ngẫu nhiên khác.
- * Khi tìm được ô trống, AI gán $\$v0 \leftarrow x$, $\$v1 \leftarrow y$ để thực hiện nước đi và in ra chuỗi strP14 để xác nhận.

- **Vị trí:** AI_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** $\$v0$, $\$v1$ – tọa độ x và y của nước đi được chọn.
- **Tác động đến biến toàn cục:** board, SIZE, current_player.
- **Thanh ghi sử dụng:** $\$t0$ – $\$t9$, $\$s0$ – $\$s7$, $\$a0$ – $\$a3$, $\$v0$ – $\$v1$, $\$sp$

25. Thực hiện di chuyển của AI (print_ai_move)

- **Tên hàm:** print_ai_move.
- **Chức năng:** Hiển thị nước đi vừa thực hiện của AI ra console. Macro này được gọi sau khi AI thực hiện một nước cờ. Nó lấy tọa độ nước đi của AI từ hai biến toàn cục ai_move_x và ai_move_y. Sau đó, in ra thông báo theo định dạng: AI plays: (x,y).
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.

- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** Không.
- **Thanh ghi sử dụng:** \$t5-\$t6, \$a0, \$v0

17. Hoàn tác trước lượt của AI (ask_undo_before_ai_move)

- **Tên hàm:** ask_undo_before_ai_move.
- **Chức năng:** Hỏi người chơi xem họ có muốn undo trước khi đến lượt AI hay không. Nếu người chơi đồng ý thì gõ "y", không thì gõ "n", nếu nhập khác thì sẽ in thông báo lỗi và yêu cầu nhập lại.
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** \$v0 (0 nếu người chơi muốn undo, 1 nếu không).
- **Tác động đến biến toàn cục:** Không.
- **Thanh ghi sử dụng:** \$t0-\$t1, \$a0, \$v0

27. Hoàn tác nước đi cuối cùng (undo_last_move)

- **Tên hàm:** undo_last_move.
- **Chức năng:** Hoàn tác (undo) nước đi cuối cùng và chuyển lượt lại cho người chơi trước đó.
 - Hàm bắt đầu bằng việc đọc lại tọa độ nước đi cuối cùng từ hai biến last_move_x và last_move_y.
 - Nếu biến undo_flag đã được đặt (nghĩa là vừa mới undo trước đó), hàm sẽ chuyển đến undo_too_much để thông báo rằng người chơi chỉ được undo một lần mỗi lượt.
 - Nếu chưa từng undo trong lượt này, nhưng giá trị last_move_x < 0 (không có nước nào để undo), thì thoát ngay (undo_done).
 - Ngược lại, hàm sẽ tính toán địa chỉ tương ứng của ô (x, y) trên mảng board và xóa nước đi tại ô đó (gán bằng 0).

- Sau đó: Giảm biến đếm `move_count` xuống 1. Đặt lại người chơi hiện tại bằng `last_player` (người vừa đi). Reset các biến `last_move_x`, `last_move_y`, `last_player` để không thể undo tiếp lần nữa. Đặt `undo_flag = 1` để báo rằng đã undo xong trong lượt này. Gọi các macro `update_player_time` và `check_timeout` để xử lý giới hạn thời gian. Nếu người chơi hết thời gian trong lúc undo, sẽ hiển thị thông báo timeout và xử lý thua cuộc (`timeout_game`). Cuối cùng, ghi thông báo "Last move undone." vào `current_message` để in ra màn hình trong lượt kế tiếp.

- **Vị trí:** `Console_part.mac`.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** `last_move_x`, `last_move_y`, `last_player`, `current_player`, `board`, `move_count`, `undo_flag`, `current_message`
- **Thanh ghi sử dụng:** `$t0-$t7`, `$s0`, `$s7`, `$a0`, `$v0`

28. Kiểm tra chuỗi thắng theo hướng xác định (`check_dir`)

- **Tên hàm:** `check_dir`.
- **Chức năng:** Kiểm tra xem tại vị trí vừa đánh có đủ số lượng quân thắng (`NUM_TO_WIN`) liên tiếp theo một hướng xác định (`dx`, `dy`) hay không. Nếu có, ghi lại tọa độ điểm đầu và điểm cuối của chuỗi thắng.
 - Hàm bắt đầu bằng cách khởi tạo biến đếm `count = 1` cho nước đi hiện tại, và bước kiểm tra `step = 1`.
 - Vòng lặp `forward_loop` được sử dụng để kiểm tra theo hướng thuận (`dx`, `dy`). Tại mỗi bước, hàm tính toán vị trí mới, kiểm tra nếu còn nằm trong phạm vi bàn cờ, rồi kiểm tra xem ô mới có cùng quân với người chơi hiện tại không. Nếu có, tiếp tục tăng đếm; nếu không, thoát khỏi vòng lặp.
 - Sau khi kết thúc kiểm tra hướng thuận, tọa độ điểm cuối chuỗi thắng được lưu vào `win_end_x` và `win_end_y`.
 - Tương tự, vòng lặp `backward_loop` thực hiện kiểm tra theo hướng ngược lại (`-dx`, `-dy`) để mở rộng chuỗi.
 - Nếu tìm được đủ số lượng quân liên tiếp, hàm lưu lại tọa độ điểm đầu chuỗi thắng vào `win_start_x` và `win_start_y`.

- Nếu tổng số lượng quân liên tiếp lớn hơn hoặc bằng NUM_TO_WIN, gán \$v0 = 1 để báo có thắng, ngược lại gán \$v0 = 0.

- **Vị trí:** Console_part.mac.
- **Tham số:** \$x, \$y – tọa độ nước đi cuối; \$player – người chơi hiện tại; \$dx, \$dy – hướng kiểm tra.
- **Giá trị trả về:** \$v0 = 1 nếu có chuỗi thắng, ngược lại \$v0 = 0.
- **Tác động đến biến toàn cục:** win_start_x, win_start_y, win_end_x, win_end_y.
- **Thanh ghi sử dụng:** \$t0–\$t9, \$s1, \$s2, \$s5, \$v0, và các thanh ghi đầu vào (\$x, \$y, \$player, \$dx, \$dy).

29. Kiểm tra chiến thắng sau khi đi (check_win)

- **Tên hàm:** check_win.
- **Chức năng:** Kiểm tra xem nước đi hiện tại có dẫn đến chiến thắng hay không bằng cách dò theo bốn hướng chính và lưu lại tọa độ đường thắng nếu tìm được.
 - Hàm bắt đầu bằng cách lưu các tham số đầu vào (\$x, \$y, \$player) vào các thanh ghi \$a0, \$a1, \$a2.
 - Lần lượt kiểm tra bốn hướng chính: ngang, dọc, chéo chính (\), và chéo phụ (/) bằng cách gọi macro check_dir với các giá trị dx, dy tương ứng.
 - Nếu bất kỳ lời gọi check_dir nào trả về \$v0 = 1, nghĩa là tìm thấy chuỗi thắng, thì nhảy đến nhãn win_found và đặt \$v0 = 1.
 - Nếu không có chuỗi thắng nào được phát hiện sau cả bốn lần kiểm tra, gán \$v0 = 0.
- **Vị trí:** Console_part.mac.
- **Tham số:** \$x, \$y – tọa độ nước đi cuối; \$player – người chơi hiện tại.
- **Giá trị trả về:** \$v0 = 1 nếu phát hiện chuỗi thắng, ngược lại \$v0 = 0.
- **Tác động đến biến toàn cục:** Gián tiếp qua check_dir – win_start_x, win_start_y, win_end_x, win_end_y.
- **Thanh ghi sử dụng:** \$s3, \$s4, \$a0–\$a2, \$v0, và các thanh ghi đầu vào (\$x, \$y, \$player).

30. Kiểm tra ván hòa khi đầy bàn cờ (check_tie)

- **Tên hàm:** check_tie.
- **Chức năng:** Kiểm tra xem bàn cờ đã đầy và không còn ô trống nào hay chưa, từ đó xác định ván đấu kết thúc với kết quả hòa.
 - Hàm bắt đầu bằng cách duyệt qua toàn bộ các ô của bàn cờ bằng hai vòng lặp lồng nhau: tie_outer (duyet theo hàng) và tie_inner (duyet theo cột).
 - Với mỗi ô tại vị trí (row, col), hàm tính địa chỉ tương ứng trong mảng board và nạp giá trị tại đó.
 - Nếu phát hiện bất kỳ ô nào có giá trị bằng 0 (tức là còn trống), hàm lập tức kết luận chưa hòa và nhảy đến nhãn not_tie để gán \$v0 = 0.
 - Nếu toàn bộ bàn cờ đều có giá trị khác 0 (đều đã đi), hàm gán \$v0 = 1 để báo ván đấu hòa.
- **Vị trí:** Console_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** \$v0 = 1 nếu ván đấu hòa, ngược lại \$v0 = 0.
- **Tác động đến biến toàn cục:** Không.
- **Thanh ghi sử dụng:** \$t0-\$t6, \$s1, \$v0.

31. Chuyển lượt chơi (switch_player)

- **Tên hàm:** switch_player.
- **Chức năng:** Chuyển quyền chơi sang người còn lại (giữa người chơi 1 và người chơi 2).
 - Hàm sử dụng biểu thức (3 - current_player) để chuyển đổi giá trị người chơi hiện tại (1 thành 2, hoặc ngược lại).
 - Đầu tiên, nạp hằng số 3 vào \$t7, sau đó lấy giá trị người chơi hiện tại từ biến toàn cục current_player vào \$s0.
 - Thực hiện phép trừ (3 - current_player) và lưu kết quả trở lại vào \$s0, đồng thời cập nhật biến toàn cục current_player.
- **Vị trí:** Console_part.mac.

- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** `current_player`.
- **Thanh ghi sử dụng:** `$t7`, `$s0`.

32. Cập nhật điểm số sau mỗi ván đấu (`update_score`)

- **Tên hàm:** `update_score`.
- **Chức năng:** Cập nhật điểm số của người chơi hoặc số ván hòa sau mỗi trận đấu.
 - Hàm nhận vào tham số `$tie`, giá trị 1 nếu ván đấu hòa và 0 nếu có người thắng cuộc.
 - Nếu là ván hòa, tăng biến toàn cục `ties` thêm 1.
 - Nếu không hòa, kiểm tra giá trị `current_player` để xác định người thắng:
 - * Nếu người chơi hiện tại là 1, tăng `player1_score`.
 - * Ngược lại, tăng `player2_score`.
- **Vị trí:** `Console_part.mac`.
- **Tham số:** `$tie` – chỉ định ván đấu có hòa không (1: hòa, 0: không hòa).
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** `player1_score`, `player2_score`, `ties`.
- **Thanh ghi sử dụng:** `$t2`, `$s0`.

33. Hiển thị thông báo người chiến thắng (`display_win_message`)

- **Tên hàm:** `display_win_message`.
- **Chức năng:** Hiển thị thông báo kết quả khi một người chơi chiến thắng, bao gồm số hiệu người chơi và số lượt đi trong ván đấu.
 - In chuỗi `strPlayer` để báo bắt đầu thông báo thắng.
 - In số hiệu người chơi thắng được truyền vào qua tham số `$player`.
 - In chuỗi `strWins` để hoàn tất câu thông báo chiến thắng.

- Tiếp tục in số lượt đi của ván đấu, lấy từ biến toàn cục `move_count`.
- Cuối cùng, in chuỗi `strMoves` để kết thúc dòng thông báo.

- **Vị trí:** `Console_part.mac`.
- **Tham số:** `$player` – người chơi chiến thắng (1 hoặc 2).
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** `move_count`.
- **Thanh ghi sử dụng:** `$s7`, `$a0`, `$v0`, và `$player`.

34. Hiển thị tọa độ đường thắng (`display_win_line`)

- **Tên hàm:** `display_win_line`.
- **Chức năng:** Hiển thị tọa độ điểm đầu và điểm cuối của chuỗi thắng trên màn hình console.
 - In ký tự khoảng trắng đầu dòng để làm đẹp định dạng.
 - In chuỗi `strWinLine` nhằm bắt đầu thông báo đường thắng.
 - In lần lượt các giá trị `win_start_x`, `win_start_y`, `win_end_x`, `win_end_y`, xen kẽ với các ký tự định dạng như `strComma`, `strTo`, `strCloseParen`.
 - Cuối cùng, in ký tự xuống dòng từ chuỗi `strNewLine`.
- **Vị trí:** `Console_part.mac`.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** `win_start_x`, `win_start_y`, `win_end_x`, `win_end_y`.
- **Thanh ghi sử dụng:** `$a0`, `$v0`.

35. Hiển thị thông báo hòa ván (`display_tie_message`)

- **Tên hàm:** `display_tie_message`.
- **Chức năng:** Hiển thị thông báo hòa ván đấu kết thúc với kết quả hòa, kèm theo số lượt đi đã diễn ra.
 - In chuỗi `strTie` để hiển thị nội dung thông báo hòa.
 - In số lượt đi từ biến toàn cục `move_count`.
 - In tiếp chuỗi `strMoves` để bổ sung đơn vị cho số lượt.
 - Cuối cùng, in ký tự xuống dòng thông qua chuỗi `strNewLine`.
- **Vị trí:** `Console_part.mac`.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** `move_count`.
- **Thanh ghi sử dụng:** `$a0`, `$v0`.

36. Hiển thị bảng điểm hiện tại (`display_score`)

- **Tên hàm:** `display_score`.
- **Chức năng:** Hiển thị bảng điểm hiện tại của hai người chơi và số ván hòa ra màn hình console.
 - In tiêu đề bảng điểm từ chuỗi `strScoreTitle`.
 - In nhãn `strPlayer1Csl` và sau đó in số điểm của người chơi 1 từ biến `player1_score`.
 - In nhãn `strPlayer2Csl` và sau đó in số điểm của người chơi 2 từ biến `player2_score`.
 - In nhãn `strTiesCsl` và sau đó in số ván hòa từ biến `ties`.
 - Kết thúc bằng việc in chuỗi `strDot` và ký tự xuống dòng từ `strNewLine`.
- **Vị trí:** `Console_part.mac`.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** `player1_score`, `player2_score`, `ties`.
- **Thanh ghi sử dụng:** `$a0`, `$v0`.

37. Ghi một ký tự vào file kết quả (write__buffer)

- **Tên hàm:** write__buffer.
- **Chức năng:** Ghi một ký tự đơn vào file kết quả bằng cách lưu ký tự vào bộ đệm và gọi syscall ghi file.
 - Đầu tiên, ký tự đầu vào (dưới dạng mã ASCII) được ghi vào biến inputBuffer.
 - Thiết lập các tham số syscall: \$a0 là bộ định danh file (file descriptor) đã được lưu trong \$s6, \$a1 là địa chỉ buffer, và \$a2 = 1 để ghi một byte.
 - Gọi syscall số 15 để thực hiện ghi ký tự ra file.
- **Vị trí:** File_part.mac.
- **Tham số:** \$char__ascii – mã ASCII của ký tự cần ghi.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** inputBuffer.
- **Thanh ghi sử dụng:** \$s6, \$a0–\$a2, \$v0, \$char__ascii.

38. Ghi một số nguyên vào file (write__number)

- **Tên hàm:** write__number.
- **Chức năng:** Ghi một số nguyên không âm từ 0 đến 99999 dưới dạng từng chữ số vào file (dưới dạng ký tự ASCII).
 - Đầu tiên, lưu tạm giá trị ban đầu của tham số \$number vào stack để phục hồi sau khi ghi xong.
 - Hàm sử dụng chia lấy phần nguyên và phần dư để tách từng chữ số từ hàng chục nghìn đến hàng đơn vị.
 - Mỗi chữ số sau khi tách sẽ được chuyển đổi sang mã ASCII bằng cách cộng thêm 48, rồi ghi vào file thông qua macro write__buffer.
 - Đối với các chữ số 0 đứng trước (ví dụ như số 005), hàm đảm bảo in đủ số chữ số bằng cách ghi thêm các ký tự '0' thủ công trong các trường hợp đặc biệt (x000x, x00xx, x0xxx...).
 - Sau khi hoàn tất, hàm phục hồi lại giá trị ban đầu của \$number từ stack.

- **Vị trí:** File_part.mac.
- **Tham số:** \$number – số nguyên cần ghi (giới hạn từ 0 đến 99999).
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** Gián tiếp sử dụng inputBuffer thông qua write_buffer.
- **Thanh ghi sử dụng:** \$t1-\$t2, \$sp, \$number.

39. Ghi nhiều ký tự khoảng trắng vào file (write_spaces)

- **Tên hàm:** write_spaces.
- **Chức năng:** Ghi một số lượng ký tự khoảng trắng (space) liên tiếp vào file.
 - Khởi tạo ký tự khoảng trắng bằng mã ASCII 32 và lưu vào thanh ghi \$t8.
 - Sử dụng biến đếm \$t9 để theo dõi số khoảng trắng đã ghi.
 - Lặp lại quá trình gọi write_buffer(\$t8) cho đến khi đủ số lượng được yêu cầu bởi tham số \$count.
- **Vị trí:** File_part.mac.
- **Tham số:** \$count – số lượng khoảng trắng cần ghi.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** Gián tiếp qua inputBuffer thông qua macro write_buffer.
- **Thanh ghi sử dụng:** \$t8, \$t9, \$count.

40. Ghi ký tự xuống dòng vào file (write_newline)

- **Tên hàm:** write_newline.
- **Chức năng:** Ghi ký tự xuống dòng (newline – \n) vào file kết quả.
 - Thiết lập mã ASCII 10 (ký tự newline) vào thanh ghi \$t8.
 - Gọi macro write_buffer(\$t8) để thực hiện ghi ký tự này vào file.
- **Vị trí:** File_part.mac.
- **Tham số:** Không.

- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** Gián tiếp thông qua `inputBuffer`.
- **Thanh ghi sử dụng:** `$t8`.

41. Ghi lịch sử hành động vào bộ nhớ (`record_action_history`)

- **Tên hàm:** `record_action_history`.
- **Chức năng:** Ghi lại một hành động trong mảng `action_history`, dùng để lưu vết trận đấu bao gồm nước đi, undo, timeout hoặc đầu hàng.
 - Hàm chỉ ghi hành động nếu số lượng hành động hiện tại (`action_count`) chưa vượt quá giới hạn cho phép (10000).
 - Tính toán địa chỉ của ô tiếp theo trong mảng `action_history` dựa trên chỉ số `action_count`.
 - Ghi loại hành động (`$type`) và người thực hiện hành động (`$player`) vào hai byte đầu tiên.
 - Nếu hành động là một nước đi thông thường (`$type = 0`), ghi thêm tọa độ `$x`, `$y`. Ngược lại, ghi giá trị -1 cho cả hai tọa độ để biểu thị hành động đặc biệt (undo, timeout, đầu hàng).
 - Tăng biến đếm `action_count` sau khi lưu thành công.
- **Vị trí:** `File_part.mac`.
- **Tham số:**
 - `$type` – loại hành động (0: đi quân, 1: undo, 2: timeout, 3: đầu hàng)
 - `$player` – người chơi thực hiện hành động (1 hoặc 2)
 - `$x`, `$y` – tọa độ quân cờ (chỉ dùng nếu `$type = 0`)
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** `action_history`, `action_count`.
- **Thanh ghi sử dụng:** `$t2–$t3`, `$t6–$t8`, `$type`, `$player`, `$x`, `$y`.

42. Ghi toàn bộ lịch sử hành động vào file (save_action_history)

- **Tên hàm:** save_action_history.
- **Chức năng:** Ghi toàn bộ lịch sử các hành động trong trận đấu (nước đi, undo, đầu hàng, timeout) vào file kết quả.
 - Mở đầu bằng cách in tiêu đề “Move History” vào file, sau đó xuống dòng.
 - Nếu không có hành động nào (action_count = 0), thoát ngay.
 - Duyệt từng hành động trong mảng action_history, mỗi hành động gồm 4 byte: type, player, x, y.
 - Với mỗi hành động hợp lệ (player = 1 hoặc 2):
 - * Tính số thứ tự và in ra (căn lề bằng khoảng trắng).
 - * Ghi người chơi thực hiện hành động.
 - * Nếu là nước đi (type = 0), ghi cặp tọa độ (x, y).
 - * Nếu là undo, surrender, hoặc timeout, in các chuỗi tương ứng (strUndoHist, strSurrHist, strTimeoutHist).
 - Mỗi hành động được in trên một dòng riêng, ngoại trừ surrender hoặc timeout sẽ kết thúc ngay phần ghi lịch sử.
- **Vị trí:** File_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** action_history, action_count.
- **Thanh ghi sử dụng:** \$t0–\$t9, \$s6, \$a0–\$a3, \$v0.

43. Ghi tiêu đề và thông tin cấu hình vào file (save_file_header)

- **Tên hàm:** save_File_header.
- **Chức năng:** Ghi tiêu đề ban đầu, cấu hình trò chơi và thông tin đối thủ vào file kết quả. Nội dung thay đổi tùy thuộc vào việc đây là ván đầu tiên hay các ván tiếp theo.
 - Nếu đây là ván đầu tiên (game_count = 1), mở file ở chế độ ghi mới và in ASCII Art tiêu đề cùng dòng phân cách.

- Nếu không, mở file ở chế độ ghi tiếp (append).
 - In tiêu đề “Game Info” kèm số thứ tự ván đấu hiện tại bằng macro `write_number`.
 - Ghi thông tin chế độ chơi: kiểm tra xem có phải chế độ mặc định không bằng cách so sánh các thông số `SIZE`, `NUM_TO_WIN`, `TIME_LIMIT`, `ADD_TIME`.
 - * Nếu đúng cấu hình mặc định (15x15, 5 thắng, 600 giây, không thêm giờ), in “Default”.
 - * Nếu khác, in “Custom” và ghi thông số cấu hình chi tiết: Size, Win, Time, AddTime, và loại bàn cờ (Blank hoặc Puzzle).
 - Ghi thông tin đối thủ: nếu đấu với AI, ghi “AI (Player X)”, ngược lại ghi “Human”.
 - Cuối cùng, xuống dòng để kết thúc phần header.
- **Vị trí:** `File_part.mac`.
 - **Tham số:** Không.
 - **Giá trị trả về:** Không.
 - **Tác động đến biến toàn cục:** `game_count`, `SIZE`, `NUM_TO_WIN`, `TIME_LIMIT`, `ADD_TIME`, `game_mode`, `puzzle_type`, `game_opponent`, `human_side`.
 - **Thanh ghi sử dụng:** `$t0-$t1`, `$t3-$t4`, `$s6`, `$a0-$a3`, `$v0`.

44. Ghi chỉ số cột và viền trên cùng của bàn cờ (`save_board_col`)

- **Tên hàm:** `save_board_col`.
- **Chức năng:** Ghi dòng tiêu đề thể hiện chỉ số cột của bàn cờ và vẽ đường viền trên cùng (top border) cho bảng trò chơi vào file kết quả.
 - Bắt đầu bằng cách ghi 8 ký tự khoảng trắng để canh lề.
 - Duyệt qua tất cả cột từ 0 đến `SIZE - 1`:
 - * In chỉ số cột bằng macro `write_number`.
 - * Thêm khoảng trắng tương ứng: 5 khoảng trắng cho số 1 chữ số, 4 cho số 2 chữ số.
 - Sau khi in hết tiêu đề cột, xuống dòng.
 - Tiếp theo, in đường viền trên:

- * Ghi ký tự '+' làm góc trái.
- * Với mỗi cột, ghi 5 ký tự '-' đại diện cho chiều ngang ô, sau đó ghi thêm dấu '+' phân cách giữa các cột.
- * Kết thúc bằng dấu '+' ở cuối viên và xuống dòng.

- **Vị trí:** File_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** SIZE.
- **Thanh ghi sử dụng:** \$t0, \$t8, \$s1, \$a0, \$a3.

45. Ghi một hàng của bàn cờ vào file (save_board_row)

- **Tên hàm:** save_board_row.
- **Chức năng:** Ghi một hàng của bàn cờ hiện tại vào file, bao gồm chỉ số hàng, nội dung từng ô và đường viền phân cách nếu cần.
 - In chỉ số hàng: nếu là số 1 chữ số thì canh lề bằng 2 khoảng trắng, nếu là 2 chữ số thì 1 khoảng trắng.
 - Viết ký tự viền trái '|'.
 - Duyệt từng cột trong hàng:
 - * Tính vị trí của ô trên mảng board và đọc giá trị ô.
 - * Tùy giá trị:
 - 0: ô trống → hiển thị dấu chấm '.' ở giữa.
 - 1: người chơi 1 → hiển thị ký hiệu [X].
 - 2: người chơi 2 → hiển thị ký hiệu (O).
 - * Thêm khoảng trắng trước/sau ký hiệu để căn giữa ô (rộng 5 ký tự), và in dấu phân cách '|'.
 - Sau khi in xong một hàng, xuống dòng.
 - Nếu không phải hàng cuối cùng, in đường phân cách giữa các hàng bằng các dấu '-' và dấu '+' tương ứng.
- **Vị trí:** File_part.mac.

- **Tham số:** \$row_num – số thứ tự của hàng cần ghi.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** board, SIZE.
- **Thanh ghi sử dụng:** \$t0–\$t8, \$s1, \$a3, \$row_num.

46. Ghi một hàng bàn cờ khởi tạo vào file (save_initial_board_row)

- **Tên hàm:** save_initial_board_row.
- **Chức năng:** Ghi một hàng của bàn cờ khởi tạo (ban đầu) vào file kết quả, bao gồm số hàng, nội dung từng ô và đường phân cách nếu cần.
 - In chỉ số hàng: nếu là số 1 chữ số thì canh lề bằng 2 khoảng trắng, nếu là 2 chữ số thì chỉ 1 khoảng trắng.
 - In ký tự viền trái '|’.
 - Duyệt từng cột trong hàng:
 - * Tính vị trí ô tương ứng trong mảng init_board và đọc giá trị tại ô.
 - * Tùy theo giá trị:
 - 0: ô trống → hiển thị dấu chấm '.'.
 - 1: người chơi 1 → hiển thị ký hiệu [X].
 - 2: người chơi 2 → hiển thị ký hiệu (O).
 - * Căn giữa biểu tượng trong ô rộng 5 ký tự bằng khoảng trắng.
 - * In ký tự '|' sau mỗi ô để tạo đường kẻ giữa các ô.
 - Sau khi in xong hàng, xuống dòng.
 - Nếu không phải hàng cuối cùng, in đường viền phân cách giữa các hàng bằng các dấu '-' và '+' tương ứng.
- **Vị trí:** File_part.mac.
- **Tham số:** \$row_num – số thứ tự của hàng cần ghi.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** init_board, SIZE.
- **Thanh ghi sử dụng:** \$t0–\$t8, \$s1, \$a3, \$row_num.

47. Ghi đường viền dưới cùng của bàn cờ vào file (save_bottom_border)

- **Tên hàm:** save_bottom_border.
- **Chức năng:** Ghi đường viền dưới cùng của bàn cờ vào file kết quả, sử dụng các ký tự '-' và '+' để tạo cấu trúc bảng.
 - Ghi 5 khoảng trắng để căn lề bên trái.
 - Bắt đầu đường viền bằng ký tự '+'.
 - Duyệt qua từng cột trong bảng:
 - * Ghi 5 ký tự '-' tương ứng với chiều rộng của mỗi ô.
 - * Nếu chưa phải cột cuối cùng, thêm dấu '+' để phân cách giữa các ô.
 - Sau khi kết thúc cột cuối, thêm dấu '+' kết thúc và xuống dòng.
- **Vị trí:** File_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** SIZE.
- **Thanh ghi sử dụng:** \$t0, \$t8, \$s1, \$a3.

48. Ghi bàn cờ khởi tạo vào file (save_initial_board)

- **Tên hàm:** save_initial_board.
- **Chức năng:** Ghi toàn bộ trạng thái bàn cờ khởi tạo (trong chế độ Puzzle) vào file kết quả, bao gồm tiêu đề cột, nội dung từng hàng và đường viền.
 - Gọi macro save_board_col để ghi dòng tiêu đề chỉ số cột và đường viền trên cùng.
 - Duyệt qua từng hàng của bàn cờ từ hàng 0 đến SIZE - 1, gọi macro save_initial_board_row để ghi nội dung từng hàng.
 - Sau khi ghi toàn bộ các hàng, gọi save_bottom_border để vẽ đường viền dưới cùng.
 - Kết thúc bằng ký tự xuống dòng để phân tách phần tiếp theo của file.
- **Vị trí:** File_part.mac.
- **Tham số:** Không.

- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** `init_board`, `SIZE`.
- **Thanh ghi sử dụng:** `$t0`, `$s1`.

49. Ghi thời gian đã chơi của hai người chơi (`save_time_played`)

- **Tên hàm:** `save_time_played`.
- **Chức năng:** Tính toán và ghi lại tổng thời gian đã sử dụng của mỗi người chơi vào file kết quả. Hàm này xử lý cả hai chế độ: có giới hạn thời gian và thời gian vô hạn.
 - Kiểm tra nếu thời gian giới hạn (`TIME_LIMIT`) là âm, thì xem như chế độ thời gian vô hạn:
 - * Ghi chuỗi Infinite cho cả hai người chơi.
 - Nếu là chế độ giới hạn thời gian:
 - * Tính số lượt đi của mỗi người chơi từ `move_count`.
 - * Với mỗi người chơi:
 - Tính tổng thời gian cấp phát ban đầu bằng: `TIME_LIMIT + ADD_TIME` * số lượt.
 - Lấy thời gian thực tế đã chơi bằng: tổng thời gian ban đầu trừ thời gian còn lại.
 - In ra dòng thông tin và thời gian đã sử dụng (tính bằng giây).
- **Vị trí:** `File_part.mac`.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** `TIME_LIMIT`, `ADD_TIME`, `move_count`, `player1_time`, `player2_time`.
- **Thanh ghi sử dụng:** `$t0–$t6`, `$t9`, `$s5–$s7`, `$a0–$a2`, `$v0`.

50. Ghi kết quả chiến thắng vào file (save_win_result)

- **Tên hàm:** save_win_result.
- **Chức năng:** Ghi thông tin người chiến thắng, số lượt đi, lý do chiến thắng (thường, đầu hàng, hết giờ), đường thắng (nếu có), và thời gian đã chơi vào file kết quả.
 - Ghi chuỗi “Result”, người chơi chiến thắng và cụm từ “wins”.
 - Ghi số lượt đi (biến move_count) và chuỗi “moves”.
 - Kiểm tra lý do thắng:
 - * Nếu last_move_x = -1 → ghi “Win by surrender”.
 - * Nếu last_move_x = -2 → ghi “Win by timeout”.
 - * Nếu là thắng thông thường → tiếp tục ghi tọa độ đường thắng.
 - Ghi đường thắng từ các biến win_start_x, win_start_y đến win_end_x, win_end_y.
 - Gọi macro save_time_played để ghi tổng thời gian đã chơi của cả hai người chơi.
 - Xuống dòng kết thúc phần kết quả.
- **Vị trí:** File_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** current_player, move_count, last_move_x, win_start_x, win_start_y, win_end_x, win_end_y, TIME_LIMIT, ADD_TIME, player1_time, player2_time.
- **Thanh ghi sử dụng:** \$t0–\$t1, \$s6–\$s7, \$a0–\$a3, \$v0.

51. Ghi kết quả hòa vào file (save_tie_result)

- **Tên hàm:** save_tie_result.
- **Chức năng:** Ghi thông báo ván đấu kết thúc với kết quả hòa, số lượt đi và thời gian đã chơi của hai người chơi vào file kết quả.
 - Ghi khoảng trắng để canh lề, sau đó ghi chuỗi “Tie”.
 - Ghi số lượt đi từ biến move_count, tiếp theo là chuỗi “moves”.
 - Ghi thêm một khoảng trắng và gọi macro save_time_played để ghi thời gian đã chơi của cả hai người chơi.

– Kết thúc bằng ký tự xuống dòng.

- **Vị trí:** File_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** move_count, TIME_LIMIT, ADD_TIME, player1_time, player2_time.
- **Thanh ghi sử dụng:** \$a0–\$a3, \$v0, \$t0.

52. Ghi điểm số hiện tại vào file (save_score)

- **Tên hàm:** save_score.
- **Chức năng:** Ghi bảng điểm hiện tại của trận đấu vào file kết quả, bao gồm:
 - Điểm số của người chơi 1 và người chơi 2, số ván hòa (ties).
 - Trạng thái đặt lại điểm (reset_score): nếu có thì ghi "Yes", ngược lại ghi "No".
 - Kết thúc bằng dấu chấm và hai dòng xuống dòng để phân tách.

- **Vị trí:** File_part.mac.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** player1_score, player2_score, ties, reset_score.
- **Thanh ghi sử dụng:** \$t0, \$s6, \$a0–\$a3, \$v0.

53. Ghi toàn bộ kết quả trận đấu vào file (save_result)

- **Tên hàm:** save_result.
- **Chức năng:** Ghi toàn bộ dữ liệu liên quan đến trận đấu vào file kết quả, bao gồm:
 - Tiêu đề file và cấu hình trò chơi (save_File_header).
 - Bàn cờ khởi tạo (save_initial_board) và bàn cờ cuối cùng (save_board_col, save_board_row).
 - Lịch sử hành động (save_action_history) và đánh dấu kết thúc lịch sử.

- Kết quả trận đấu: nếu hòa → gọi `save_tie_result`, ngược lại gọi `save_win_result`.
- Ghi bảng điểm cuối cùng (`save_score`) và dòng phân cách.
- Đóng file kết quả bằng `syscall`.

- **Vị trí:** `File_part.mac`.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** `init_board`, `board`, `action_history`, `player1_score`, `player2_score`, `ties`, `reset_score`, `move_count`, `TIME_LIMIT`, `ADD_TIME`, `player1_time`, `player2_time`, `current_player`, `win_start_x`, `win_start_y`, `win_end_x`, `win_end_y`, `last_move_x`.
- **Thanh ghi sử dụng:** `$t0-$t1`, `$s6`, `$a0-$a2`, `$v0`.

54. Ghi thông điệp kết thúc trò chơi vào file (`save_endgame_message`)

- **Tên hàm:** `save_endgame_message`.
- **Chức năng:** Mở file kết quả ở chế độ ghi tiếp (`append`), ghi thông điệp kết thúc trò chơi với căn lề giữa, sau đó đóng file.
- **Vị trí:** `File_part.mac`.
- **Tham số:** Không.
- **Giá trị trả về:** Không.
- **Tác động đến biến toàn cục:** `s6`, `strEndGame`.
- **Thanh ghi sử dụng:** `$s6`, `$a0-$a3`, `$v0`.

6 Chạy Chương Trình Trò Chơi Gomoku

Code hàm Main:

Main program

```
# ----- MAIN PART ----- #

# Gomoku Game Implementation (5 in a rows, 15 x 15 blank board, 600 seconds time limit,
# no time added in default mode)

# Macros for complete game functionality
#include "Data_part.mac"
#include "AI_part.mac"
#include "Console_part.mac"
#include "File_part.mac"

# -----
# Main Game Loop
#
# Description: Play a match, print and save result . Then ask for replay
# Main registers: $s0 (current player), $s7 (move count), $t0 (x), $t1 (y),
#                $t2, $t3, $t4 (temp variables for different purposes),
#                $v0, $v1 (for result), $a0 (for print strings), $sp (backup)
# -----
.text
.globl main
main:
    # Display Gomoku ASCII art
    display_label
    sw $zero, current_message

    # Choose mode, board type, opponent, side
    choose_game_information

# -----
# GAME INITIALIZATION
# -----
    game_start:
        # Increase num of games
        lw $t3, game_count
```

```
    addi $t3, $t3, 1
    sw $t3, game_count

    # Initialize game
    reset_game    # Current player = 1, move count = 0

    # Create puzzle board if in puzzle type
    lw $t2, puzzle_type
    beqz $t2, no_puzzle
    generate_random_moves
    no_puzzle:
    create_initial_board

# -----
# MAIN GAME LOOP
# -----
game_loop:
    # ---- Check timeout
    # Check time before each turn
    update_player_time
    check_timeout
    beq $v0, 0, no_timeout

    # ---- Timeout case
    out_time:
        # Timeout occurred - current player loses
        set_time_to_zero

        print_board
        la $a0, strTimeout
        li $v0, 4
        syscall

        # Switch to other player who wins by timeout
        switch_player
        j timeout_game

    # ---- No timeout case
    no_timeout:
        # Print current board
```

```
print_board

# Display AI move if AI played the last turn
print_ai_move

# ---- Get player input
# Check if current player is human or AI
lw $t2, game_opponent
beqz $t2, human_turn # If game_opponent = 0 (human vs human)

# If game_opponent = 1 (human vs AI)
lw $t3, human_side
lw $s0, current_player
beq $s0, $t3, human_turn # If current player matches human side

# Otherwise it's AI's turn
j ai_turn

human_turn:
    # Reset AI last move flag
    li $t4, 0
    sw $t4, ai_last_move

    # Original human input code
    get_player_input
    sw $v0, move_x # x or 'u' or 's'
    sw $v1, move_y # y or 'u' or 's'

    # Handle special commands
    li $t2, -1 # 'u' = -1
    beq $v0, $t2, handle_undo
    li $t2, -2 # 's' = -2
    beq $v0, $t2, handle_surrender
    li $t2, -3 # Timeout = -3
    beq $v0, $t2, out_time
    j make_move

ai_turn:
    # Ask player undo before AI
    lw $s7, move_count
```

```
    beqz $s7, no_undo_before_ai_move
    ask_undo_before_ai_move
    beq $v0, 0, no_undo_before_ai_move

    # If yes, handle undo before AI, switch to player 1 turn
    switch_player
    li $t4, 1    # Undo = 1
    lw $s0, current_player
    record_action_history($t4, $s0, $zero, $zero)
    switch_player

    undo_last_move
    j game_loop

no_undo_before_ai_move:
    # AI's turn
    ai_make_move

    # Set flag that last move was by AI
    li $t4, 1
    sw $t4, ai_last_move

    update_ai_time    # 5s per move

    sw $v0, move_x    # x
    sw $v1, move_y    # y

make_move:
    # Make the move
    lw $s0, current_player
    lw $t0, move_x
    lw $t1, move_y
    make_move($t0, $t1, $s0)

    # Reset undo flag on valid move
    sw $zero, undo_flag

    beq $v0, 1, game_loop # If occupied (shouldn't happen with AI), try again

    # Record the move
```

```
li $t4, 0          # Normal = 0
lw $s0, current_player
record_action_history($t4, $s0, $t0, $t1)

lw $s7, move_count
addi $s7, $s7, 1 # Increase count
sw $s7, move_count

# Determine side to check, because AI mode not use normal
# update_player_time
determine_side_check

human_check:
    # ---- Check win and find winning line
    update_player_time
    check_timeout
    bne $v0, 0, out_time

    lw $s0, current_player
    lw $t0, move_x
    lw $t1, move_y
    check_win($t0, $t1, $s0)
    bne $v0, 0, game_over

    # ---- Check tie
    update_player_time
    check_timeout
    bne $v0, 0, out_time

    check_tie
    bne $v0, 0, tie_game

    # ---- Switch player
    update_player_time
    check_timeout
    bne $v0, 0, out_time

    switch_player
    j game_loop
```

```
ai_check:
    # ---- Check win
    check_ai_timeout
    bnez $v0, out_time

    lw $s0, current_player
    lw $t0, move_x
    lw $t1, move_y
    check_win($t0, $t1, $s0)
    bne $v0, 0, game_over

    # ---- Check tie
    check_ai_timeout
    bnez $v0, out_time

    check_tie
    bne $v0, 0, tie_game

    # ---- Switch player
    check_ai_timeout
    bnez $v0, out_time

    switch_player
    j game_loop

# -----
# SPECIAL COMMAND HANDLERS
# -----

handle_undo:
    switch_player
    li $t4, 1    # Undo = 1
    lw $s0, current_player
    record_action_history($t4, $s0, $zero, $zero)
    switch_player

    undo_last_move
    j game_loop

handle_surrender:
    li $t4, 2    # Surrender = 2
```

```
lw $s0, current_player
record_action_history($t4, $s0, $zero, $zero)

update_player_time
check_timeout
bne $v0, 0, out_time

# Store surrender message for next display
la $t4, strSurrender
sw $t4, current_message

# Current player surrender, the other win
update_player_time
switch_player
j surrender_game

# -----
# GAME END CONDITIONS
# -----
game_over:
li $t2, 0    # No tie = 0
update_score($t2)
print_board

save_result
lw $s0, current_player
display_win_message($s0)

# Check if surrender or timeout
lw $t3, last_move_x
li $t2, -1
beq $t3, $t2, show_surrender_message
li $t1, -2
beq $t3, $t2, show_timeout_message

# Normal win
display_win_line
j ask_replay_prompt

show_surrender_message:
```

```
    la $a0, strWinBySrd
    li $v0, 4
    syscall
    j ask_replay_prompt

show_timeout_message:
    la $a0, strWinByTo
    li $v0, 4
    syscall
    j ask_replay_prompt

surrender_game:
    li $t2, 0    # No tie = 0
    update_score($t2)
    print_board

    # Flag by set last_move_x = -1 (indicates surrender)
    li $t2, -1
    sw $t2, last_move_x

    # Save game information to file
    save_result

    lw $s0, current_player
    display_win_message($s0)

    la $a0, strWinBySrd
    li $v0, 4
    syscall
    j ask_replay_prompt

timeout_game:
    li $t2, 0    # No tie = 0
    update_score($t2)

    switch_player
    li $t4, 3    # Timeout = 3
    lw $s0, current_player
    record_action_history($t4, $s0, $zero, $zero)
    switch_player
```

```
# Flag by set last_move_x = -2 (indicates timeout)
li $t2, -2
sw $t2, last_move_x

# Save game infor to file
save_result

lw $s0, current_player
display_win_message($s0)

la $a0, strWinByTo
li $v0, 4
syscall
j ask_replay_prompt

tie_game:
    li $t2, 1    # Tie = 1
    update_score($t2)
    print_board

    # Save game infor to file
    save_result
    display_tie_message

# -----
# POST-GAME OPTIONS
# -----

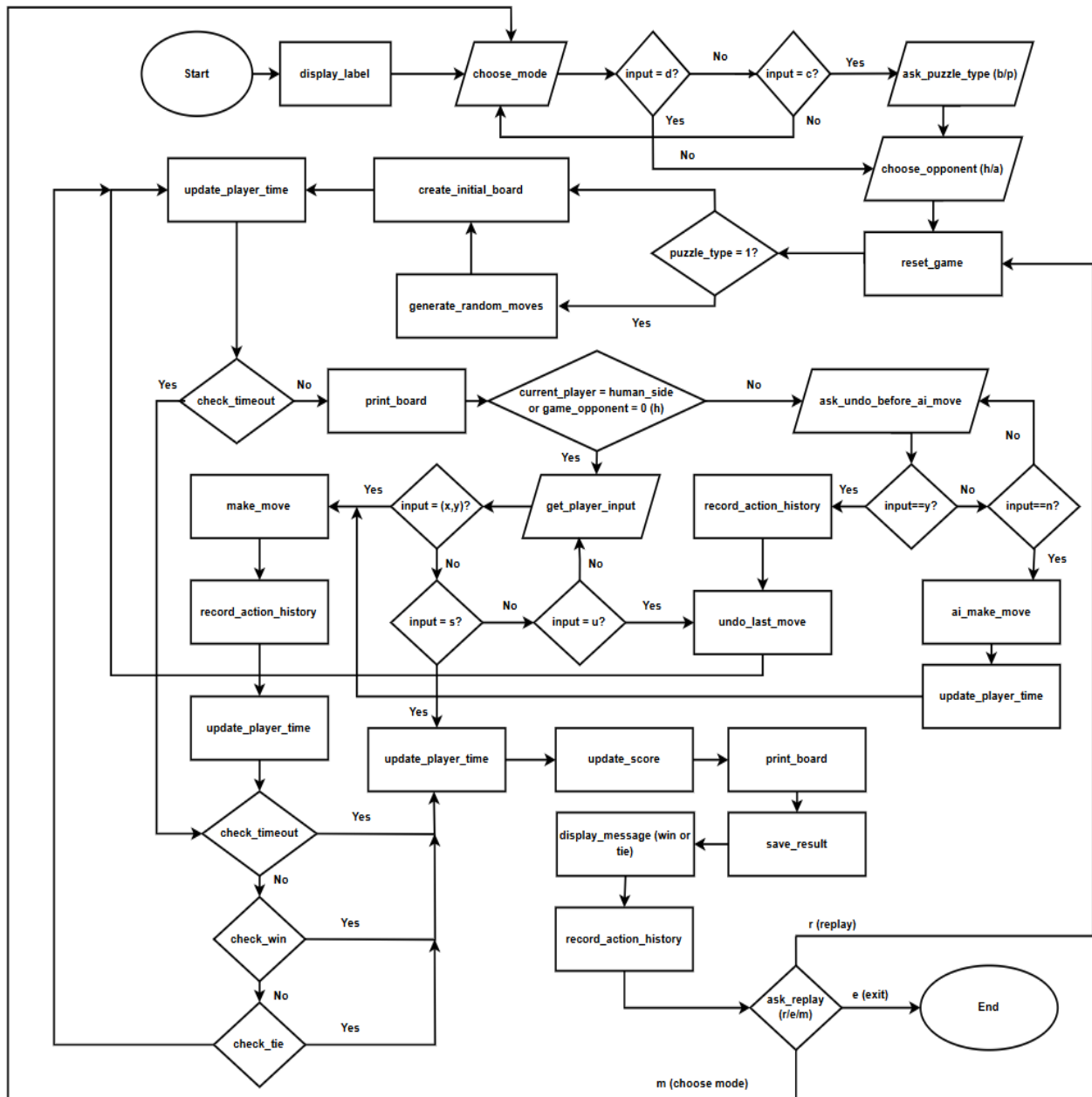
ask_replay_prompt:
    display_score
    ask_replay

    beq $v0, 1, game_start      # If yes (y), restart game with same mode
    beq $v0, 2, choose_mode_again # If mode (m), choose new mode first
    j exit                     # If no (n), exit

choose_mode_again:
    choose_game_information
    j game_start              # Then start new game
```

```
# -----  
# PROGRAM EXIT  
# -----  
exit :  
    save_endgame_message  
    la $a0, strExit  
    li $v0, 4  
    syscall  
  
# Exit  
    li $v0, 10  
    syscall
```

Flowchart:



7 Tổng Kết

Trong quá trình thực hiện đề tài, nhóm đã hoàn thành trò chơi Gomoku trên nền tảng ngôn ngữ hợp ngữ MIPS với các chức năng cơ bản và nâng cao theo yêu cầu. Dưới đây là những điểm nổi bật của đề tài:

1. Thành tựu đạt được

- **Tính năng cơ bản:**

- Hiện thực hóa thành công trò chơi Gomoku với bàn cờ kích thước 15×15 , hỗ trợ hai người chơi hoặc chơi với máy (AI).
- Kiểm tra điều kiện thắng/thua/hòa chính xác theo luật Gomoku.
- Hiển thị giao diện dòng lệnh rõ ràng, hỗ trợ nhập liệu và thông báo lỗi chi tiết.

- **Tính năng nâng cao:**

- Hỗ trợ **undo** (hoàn tác nước đi), **đầu hàng**, và **timeout** cho mỗi lượt chơi.
- AI có khả năng đánh giá nước đi hợp lý dựa trên thuật toán heuristic đơn giản.
- Lưu trữ kết quả trận đấu vào file result.txt để tiện theo dõi.
- Hỗ trợ tùy chỉnh chế độ chơi (kích thước bàn cờ, số quân thắng, thời gian lượt).

- **Tối ưu hóa:**

- Sử dụng hiệu quả thanh ghi và bộ nhớ trong MIPS để giảm độ trễ.
- Tổ chức mã nguồn modular với các file riêng biệt, dễ bảo trì và mở rộng.

2. Khó khăn và bài học kinh nghiệm

- **Quản lý thanh ghi:** Gặp khó khăn với việc ghi đè thanh ghi ngoài ý muốn. Bài học: Cần có chiến lược quản lý thanh ghi rõ ràng, lưu/phục hồi giá trị khi cần thiết.
- **Chuyển logic sang macro:** Khó debug do macro không chạy độc lập. Kinh nghiệm: Ghi chú cẩn thận và kiểm tra từng macro riêng lẻ trước khi tích hợp.
- **Xử lý hiển thị:** Canh lề bảng cờ phức tạp với các chỉ số 2 chữ số. Giải pháp: Thử nghiệm nhiều lần để tìm bố cục tối ưu.

- **Kiểm tra đầu vào:** Xử lý nhập liệu phức tạp với nhiều định dạng khác nhau. Bài học: Viết hàm kiểm tra chặt chẽ và thử nghiệm với nhiều trường hợp biên.
- **Ghi file kết quả:** Khó khăn khi làm việc với syscall file I/O. Kinh nghiệm: Đóng/mở file đúng cách và kiểm tra từng bước ghi dữ liệu.

3. Hướng phát triển

- Nâng cấp AI với thuật toán Minimax/Alpha-Beta Pruning
- Phát triển giao diện đồ họa đơn giản
- Mở rộng sang các kiến trúc khác (x86, ARM)
- Thêm các chế độ hay hơn cho game
- Thêm các giải pháp kết nối với người chơi từ xa

4. Kết luận

Đề tài đã giúp nhóm nâng cao kỹ năng lập trình Assembly, tư duy thuật toán và làm việc nhóm. Các khó khăn gặp phải trở thành bài học quý giá về debug, tối ưu hóa và quản lý dự án. Kết quả đạt được làm tiền đề cho các ứng dụng phức tạp hơn trong tương lai.

8 Tài Liệu Tham Khảo

- [1] Ho Chi Minh City University of Technology. *Phuluc_MARS*. Available at: <https://bit.ly/3zl35zc>
- [2] MIPS by imagination. *The MIPS32 Instruction Set Manual*. Available at: https://hades.mech.northwestern.edu/images/1/16/MIPS32_Architecture_Volume_II-A_Instruction_Set.pdf
- [3] https://www.codeproject.com/Articles/5375122/SharpMoku-a-Gomoku-Five-in-a-Row-_Written-in-Csharp